Member-only story

How Continuous Batching Actually Works?

4 min read · 9 hours ago

Algo Insights Following

Listen Share More

Every LLM — from Qwen to Claude to ChatGPT — produces text one token at a time. This process is slow and costly when thousands of users send requests simultaneously. In this article, we start with the basics: **attention — KV caching — chunked prefill — ragged batching**, and ultimately reach **continuous batching**, the technique that enables fast modern LLM serving.

Photo by PAUL SMITH on Unsplash

Why Does the First Token Take So Long?

If you've ever used an AI chatbot, you probably noticed:

- The **first word** takes the longest.
- After that, tokens stream at a steady tempo.

This happens because LLMs work like this:

1. Read your entire prompt
2. Produce **one** next token
3. Append that token
4. Repeat step (2) thousands of times

Each step involves billions of matrix multiplications. Serving *many* users efficiently requires careful engineering techniques. One of the most important is **continuous batching**.

To understand it, we have to start at the beginning — with **attention**.

Step 1: Attention — How Tokens Interact

Each token is converted into a d-dimensional vector.
For a sequence of 7 tokens:

```
x.shape  # [batch_size=1, seq_len=7, hidden_dim=d]
```

From `x`, we compute:

```
Q = x @ Wq
K = x @ Wk
V = x @ Wv
```

Each has shape:

```
[1, seq_len, head_dim]
```

The attention scores are:

```
scores = Q @ K.transpose(-1, -2)   # shape: [1, n, n]
```

Then a **causal mask** ensures token i can only see tokens $\leq i$:

```
scores = scores.masked_fill(mask == 0, -inf)
weights = softmax(scores, dim=-1)
```

```
output = weights @ V
```

Visually, the attention mask looks like this:

```
token0 ✓ ✓ ✓ ✓ ✓ ✓ ✓
token1   ✓ ✓ ✓ ✓ ✓ ✓
token2     ✓ ✓ ✓ ✓ ✓
...
token6           ✓
```

Each token attends only to previous ones.

Step 2: KV Cache — Why Decoding Is Fast

When generating the *next* token, naïvely we would recompute K/V for all previous tokens:

```
<BOS> I am sure this pro ject will...
 ↑     ↑  ↑     ↑      ↑   ↑    ↑
 recalculated every time x
```

Instead, we **store** all previously computed key–value pairs:

```
kv_cache.append((K_new, V_new))
```

Now each decoding step only computes Q for the *new* token → **O(n)** instead of **O(n^2)** compute.

This is the foundation for everything else.

Step 3: Chunked Prefill — Handling Long Prompts

Sometimes the initial prompt is too long to fit in GPU memory for a single pass.

Example:
GPU can process only **4 tokens** at once (`m=4`), but the prompt has **7 tokens**.

Solution: **chunked prefill**.

```
for chunk in chunks(prompt_tokens, chunk_size=4):
    Q, K, V = model(chunk)
    kv_cache.store(K, V)
```

We split the prompt and keep extending the KV cache.

Step 4: Batching — The Classic Approach and Its Problems

A simple way to serve multiple users is to put sequences into a batch:

```
batch = [
  prompt1 + [pad, pad, pad],
  prompt2 + [pad]
]
```

But this creates problems:

- Prompts must be padded to the **same length**
- If one sequence finishes early (hits `<eos>`), the GPU still computes useless tokens
- Padding grows quadratically with batch size and sequence length

This is computationally wasteful.

Step 5: Dynamic Batching — Replacing Finished Sequences

If one sequence finishes, we replace it with a new one.

But the new prompt needs prefilling (*many tokens*) while others are decoded (*one token*).
This requires adding huge padding blocks:

```
[prompt_in_decode, prompt_in_decode, ... , new_full_prompt_with_padding]
```

Often **hundreds** of padding tokens are wasted per step.

Step 6: Ragged Batching — The Breakthrough

To eliminate padding, we remove the batch dimension altogether.

Instead of:

```
[
 [token t1        ...],
 [token t2...]
]
```

We **concatenate all tokens into one long sequence:**

```
[tokens from request A] + [tokens from request B]
```

And use the *attention mask* to ensure tokens from different prompts *never see each other*:

```
mask[i][j] = 1  if token_j belongs to the same sequence as token_i
mask[i][j] = 0  otherwise
```

This is called **ragged batching**.

No padding.
No wasted compute.
Just more tokens packed together in one forward pass.

Step 7: Continuous Batching — The Final Form

Now combine everything:

KV cache: allows mixing prefill and decode stages

Chunked prefill: lets long prompts fit into the batch

Ragged batching: avoids padding and maximizes throughput

Dynamic scheduling: frees slots immediately when requests finish

A simplified algorithm:

```
while True:
    batch_tokens = []

# 1. Fill with decode-phase sequences (1 token each)
    for seq in decoding_sequences:
        batch_tokens.append(seq.next_token())
    # 2. Fill remaining capacity with prefill chunks
    while len(batch_tokens) < max_tokens:
        next_seq = prefill_queue.pop()
        chunk = next_seq.get_next_chunk()
        batch_tokens.extend(chunk)
    # 3. Construct attention mask for ragged batching
    mask = build_attention_mask(batch_tokens)
    # 4. Run one forward pass
    output = model(batch_tokens, mask, kv_cache)
```

```
        # 5. Update sequences, remove finished ones
        update_sequences(output)
```

This is **continuous batching**, the technique used in real production systems such as:

- ChatGPT
- Claude Server
- Qwen Server
- TensorRT-LLM
- vLLM

It keeps the GPU close to **100% utilization** — even with wildly different request lengths and arrival times.

Final Summary

Here's the entire story in one picture:

1. KV Caching

Reuse past keys/values → fast decoding.

2. Chunked Prefill

Break long prompts into manageable chunks.

3. Ragged Batching

Concatenate all sequences with a smart mask → zero padding waste.

4. Dynamic Scheduling

Swap finished requests with new ones instantly.

Together, they create **continuous batching**, which:

- boosts throughput dramatically
- keeps GPU fully utilized
- supports thousands of concurrent users

- reduces latency for everyone

This is the secret sauce behind modern LLM serving.

ChatGPT Qwen AI Token AI Agent

Following

Published in Coding Nexus

9.3K followers · Last published 6 hours ago

Coding Nexus is a community of developers, tech enthusiasts, and aspiring coders. Whether you're exploring the depths of Python, diving into data science, mastering web development, or staying updated on the latest trends in AI, Coding Nexus has something for you.

Following

Written by Algo Insights

3.7K followers · 6 following

Algo Insights: Unlocking the Future of Trading with Code, Data, and Intelligence. https://linktr.ee/Algo_Insights

Responses (1)

Kafkas M. Caprazli

What are your thoughts?